

# Traffic Analysis and ML models

Juan Esteban Monsalve Castrillón

## Importo las librerias a utilizar

```
In [ ]: import pandas as pd
import sklearn as sk
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
```

## Carga de datos

```
In [ ]: # #Para cargar Local
# # df = files.upload()
# data = pd.read_excel('Traffic_data.xlsx')
```

```
In [ ]: #Importe desde Google Drive
from google.colab import drive
drive.mount("/content/drive", force_remount=True)

data = pd.read_excel('/content/drive/MyDrive/Traffic_data.xlsx')
```

Mounted at /content/drive

```
In [ ]: data.head()
#Variable a predecir Traffic Situation
```

```
Out[ ]:
```

|   | Time     | Time_2        | Date | Day of<br>the<br>week | CarCount | BikeCount | BusCount | TruckCount | weather |
|---|----------|---------------|------|-----------------------|----------|-----------|----------|------------|---------|
| 0 | 00:00:00 | early morning | 10   | Tuesday               | 31       | 0         | 4        | 4          | cloudy  |
| 1 | 00:15:00 | early morning | 10   | Tuesday               | 49       | 0         | 3        | 3          | rain    |
| 2 | 00:30:00 | early morning | 10   | Tuesday               | 46       | 0         | 3        | 6          | rain    |
| 3 | 00:45:00 | early morning | 10   | Tuesday               | 51       | 0         | 2        | 5          | cloudy  |
| 4 | 01:00:00 | early morning | 10   | Tuesday               | 57       | 6         | 15       | 16         | rain    |

```
In [ ]: print(data.shape)
```

```
(2976, 10)
```

```
In [ ]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2976 entries, 0 to 2975
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Time                   2976 non-null   object
1   Time_2                 2976 non-null   object
2   Date                   2976 non-null   int64
3   Day of the week        2976 non-null   object
4   CarCount               2976 non-null   int64
5   BikeCount              2976 non-null   int64
6   BusCount               2976 non-null   int64
7   TruckCount             2976 non-null   int64
8   weather                2976 non-null   object
9   Traffic Situation      2976 non-null   object
dtypes: int64(5), object(5)
memory usage: 232.6+ KB
```

```
In [ ]: data.describe()
```

```
Out[ ]:
```

|              | Date        | CarCount    | BikeCount   | BusCount    | TruckCount   |
|--------------|-------------|-------------|-------------|-------------|--------------|
| <b>count</b> | 2976.000000 | 2976.000000 | 2976.000000 | 2976.000000 | 2.976000e+03 |
| <b>mean</b>  | 16.000000   | 68.595766   | 14.915995   | 15.278898   | 3.375539e+03 |
| <b>std</b>   | 8.945775    | 46.001420   | 12.849079   | 14.342702   | 1.833086e+05 |
| <b>min</b>   | 1.000000    | -129.000000 | -2.000000   | -1.000000   | 0.000000e+00 |
| <b>25%</b>   | 8.000000    | 19.000000   | 5.000000    | 1.000000    | 6.000000e+00 |
| <b>50%</b>   | 16.000000   | 64.000000   | 12.000000   | 12.000000   | 1.400000e+01 |
| <b>75%</b>   | 24.000000   | 107.000000  | 22.000000   | 25.000000   | 2.325000e+01 |
| <b>max</b>   | 31.000000   | 180.000000  | 70.000000   | 50.000000   | 1.000000e+07 |

## Analisis de datos

### Corrección de outliers y ceros

```
In [ ]: #Se revisaron Los datos menores a 0 y les asignamos el promedio de la columna sin t
data.loc[data['CarCount'] < 0, 'CarCount'] = data[data['CarCount'] >= 0]['CarCount'].mean()
data.loc[data['BikeCount'] < 0, 'BikeCount'] = data[data['BikeCount'] >= 0]['BikeCount'].mean()
data.loc[data['BusCount'] < 0, 'BusCount'] = data[data['BusCount'] >= 0]['BusCount'].mean()
```

```

<ipython-input-96-7f0bbc728f03>:1: FutureWarning: Setting an item of incompatible dt
ype is deprecated and will raise an error in a future version of pandas. Value '68.7
15438950555' has dtype incompatible with int64, please explicitly cast to a compatib
le dtype first.
    data.loc[data['CarCount'] < 0, 'CarCount'] = data[data['CarCount'] >= 0]['CarCoun
t'].mean()
<ipython-input-96-7f0bbc728f03>:2: FutureWarning: Setting an item of incompatible dt
ype is deprecated and will raise an error in a future version of pandas. Value '14.9
21680672268907' has dtype incompatible with int64, please explicitly cast to a compa
tible dtype first.
    data.loc[data['BikeCount'] < 0, 'BikeCount'] = data[data['BikeCount'] >= 0]['BikeC
ount'].mean()
<ipython-input-96-7f0bbc728f03>:3: FutureWarning: Setting an item of incompatible dt
ype is deprecated and will raise an error in a future version of pandas. Value '15.2
8436974789916' has dtype incompatible with int64, please explicitly cast to a compa
tible dtype first.
    data.loc[data['BusCount'] < 0, 'BusCount'] = data[data['BusCount'] >= 0]['BusCoun
t'].mean()

```

In [ ]: `data.describe()`

Out [ ]:

|              | Date        | CarCount    | BikeCount   | BusCount    | TruckCount   |
|--------------|-------------|-------------|-------------|-------------|--------------|
| <b>count</b> | 2976.000000 | 2976.000000 | 2976.000000 | 2976.000000 | 2.976000e+03 |
| <b>mean</b>  | 16.000000   | 68.715439   | 14.921681   | 15.284370   | 3.375539e+03 |
| <b>std</b>   | 8.945775    | 45.812366   | 12.845334   | 14.339596   | 1.833086e+05 |
| <b>min</b>   | 1.000000    | 6.000000    | 0.000000    | 0.000000    | 0.000000e+00 |
| <b>25%</b>   | 8.000000    | 19.750000   | 5.000000    | 1.000000    | 6.000000e+00 |
| <b>50%</b>   | 16.000000   | 64.000000   | 12.000000   | 12.000000   | 1.400000e+01 |
| <b>75%</b>   | 24.000000   | 107.000000  | 22.000000   | 25.000000   | 2.325000e+01 |
| <b>max</b>   | 31.000000   | 180.000000  | 70.000000   | 50.000000   | 1.000000e+07 |

```

In [ ]: #Conteo de datos de 0 en el Dataset
#Columnas que tienen datos con valor cero
columnas_con_ceros = data.columns[(data == 0).any()]

#Cuantos valores ceros tiene cada columna
for columna in columnas_con_ceros:
    conteo_ceros = (data[columna] == 0).sum()
    print(f"Columna - {columna}: {conteo_ceros} ceros")

```

Columna - BikeCount: 145 ceros  
 Columna - BusCount: 437 ceros  
 Columna - TruckCount: 119 ceros

## Cambio de nombre de las variables

```
In [ ]: #Se cambian las variables a sus respectivos nombres en español
data.columns = [
    'tiempo', 'tiempo_2', 'dia_mes', 'dia_semana',
    'conteo_carros', 'conteo_bicicletas', 'conteo_buses', 'conteo_camiones', 'clima'
]
```

```
In [ ]: #Se va a predecir la variable situacion_trafico
#Se elimino la variable 'tiempo' ya que nos se va a hacer un analisis temporal por
data = data[['tiempo_2', 'dia_mes', 'dia_semana',
    'conteo_carros', 'conteo_bicicletas', 'conteo_buses', 'conteo_camiones', 'clima'
]]
```

## Correlación entre la data

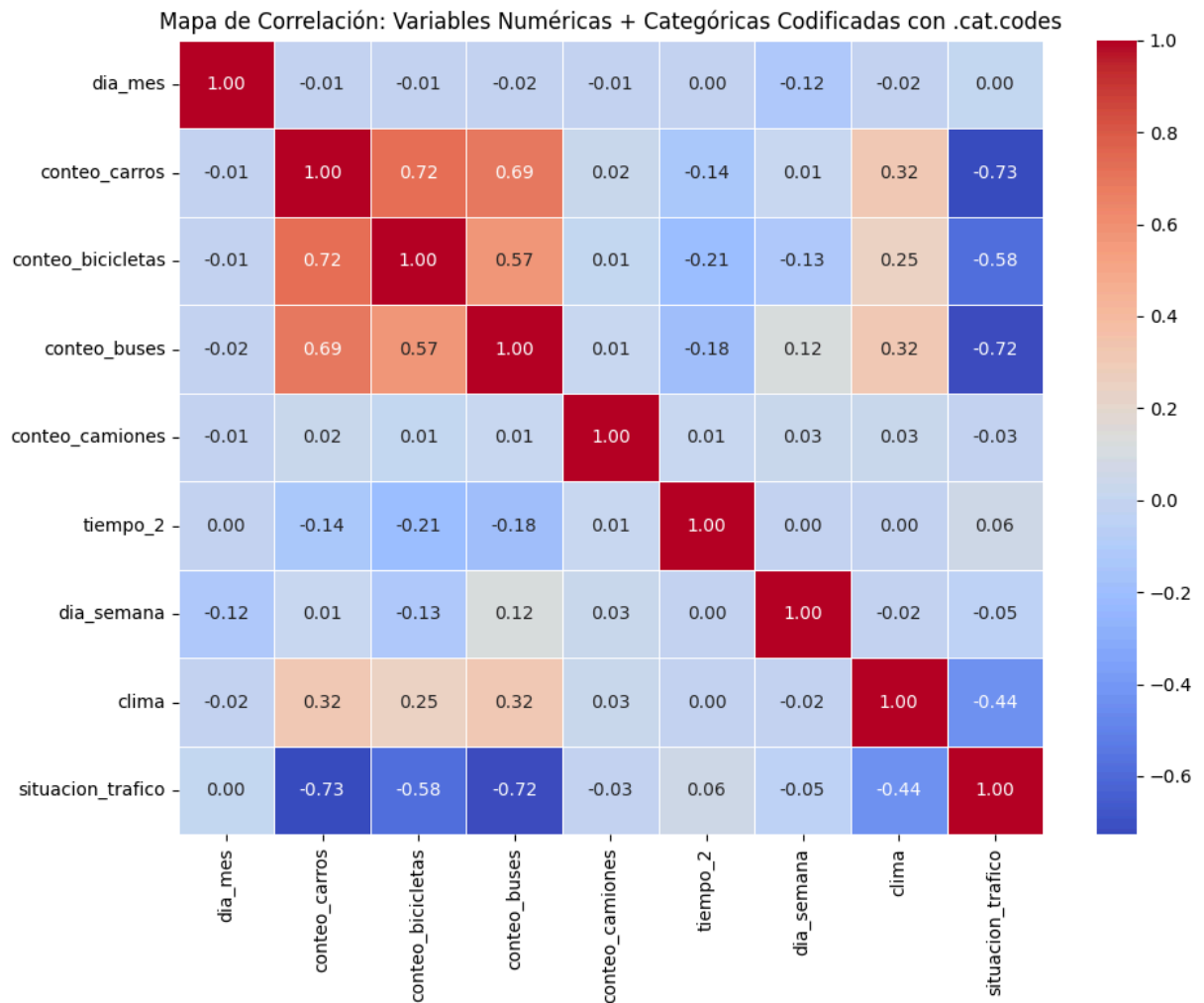
```
In [ ]: #Separo las variables en numericas y categoricas. Luego se juntan en un mismo df
variables_numericas = ['dia_mes', 'conteo_carros', 'conteo_bicicletas', 'conteo_bus
variables_categoricas = ['tiempo_2', 'dia_semana', 'clima', 'situacion_trafico']

data_subset = data[variables_numericas + variables_categoricas]

# Codificamos las variables
data_codificada = data_subset.copy()
for col in variables_categoricas:
    data_codificada[col] = data_codificada[col].astype('category').cat.codes

#Se crea la matriz de correlacion y se muestra
matriz_correlacion = data_codificada.corr()

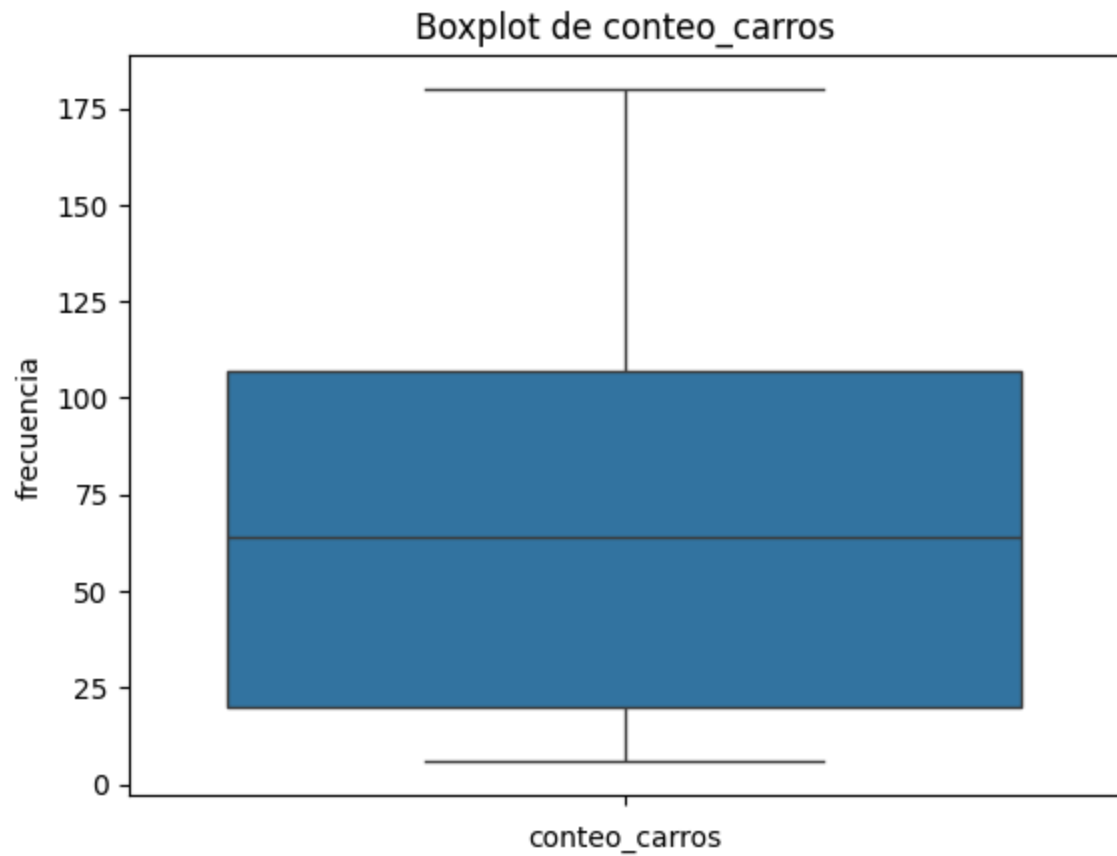
plt.figure(figsize=(10, 8))
sns.heatmap(matriz_correlacion, cmap='coolwarm', annot=True, fmt=".2f", linewidths=
plt.title("Mapa de Correlación: Variables Numéricas + Categóricas Codificadas con .
plt.tight_layout()
plt.show()
```



Se puede ver que las variables que tienen mayor conexión con la variable a analizar son 'conteo\_carros', 'conteo\_bicicletas', 'conteo\_buses', 'clima', por lo que serán las variables que introduzcamos a ambos modelos para comprobar su efectividad

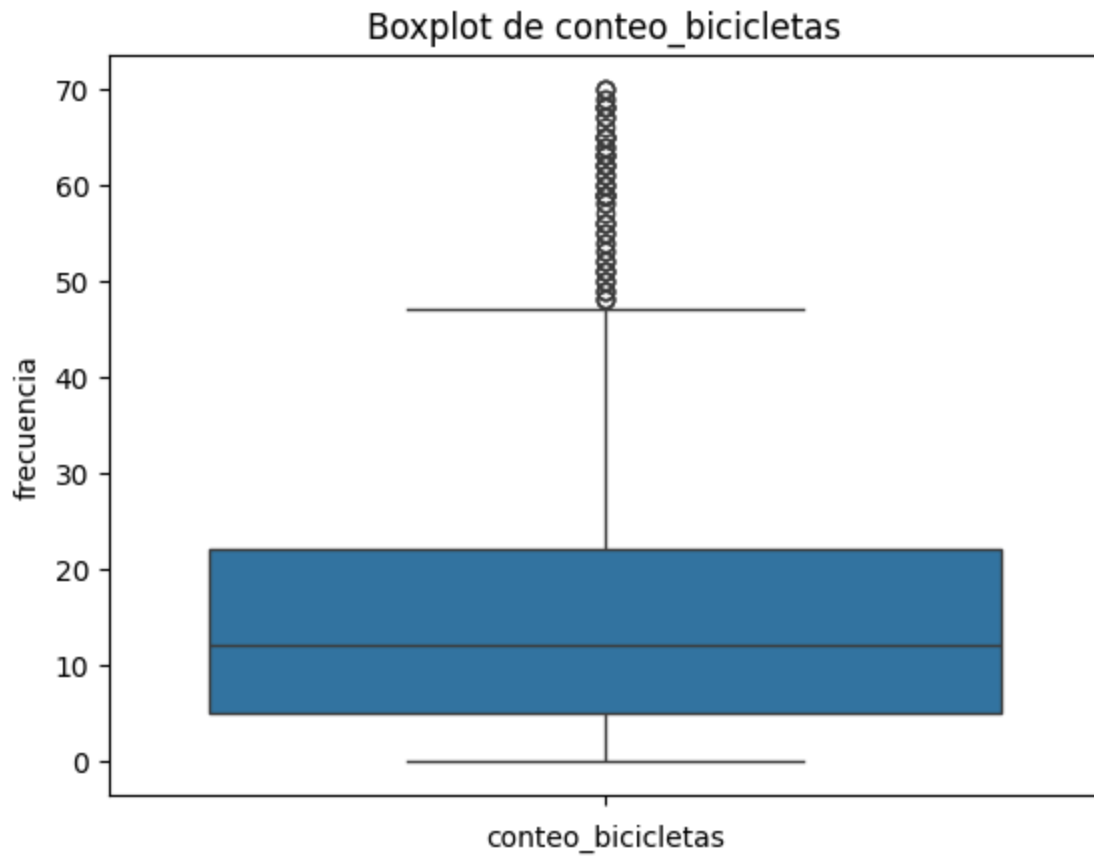
```
In [ ]: #Grafica para la variable conteo_carros
sns.boxplot(data['conteo_carros'])

plt.title('Boxplot de conteo_carros')
plt.xlabel('conteo_carros')
plt.ylabel('frecuencia')
plt.show()
```



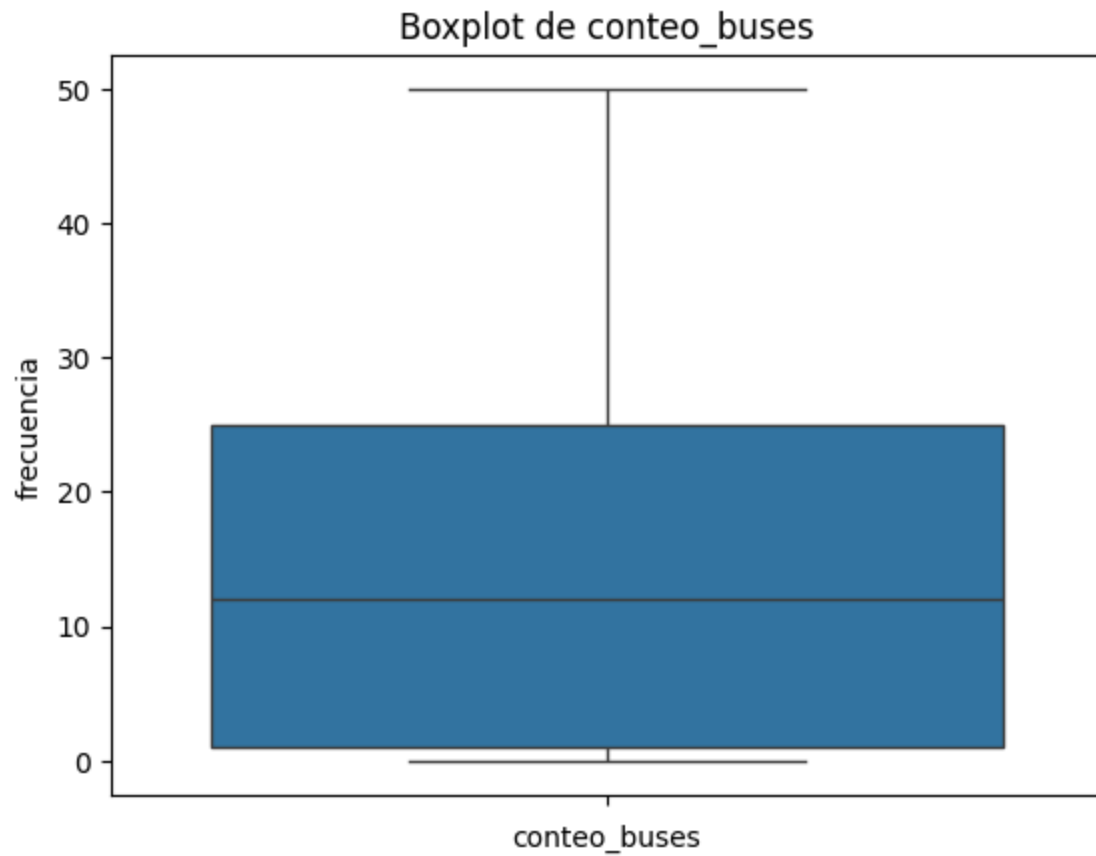
```
In [ ]: #Grafica para la variable conteo_bicicletas
sns.boxplot(data['conteo_bicicletas'])

plt.title('Boxplot de conteo_bicicletas')
plt.xlabel('conteo_bicicletas')
plt.ylabel('frecuencia')
plt.show()
```



```
In [ ]: #Grafica para la variable conteo_buses
sns.boxplot(data['conteo_buses'])

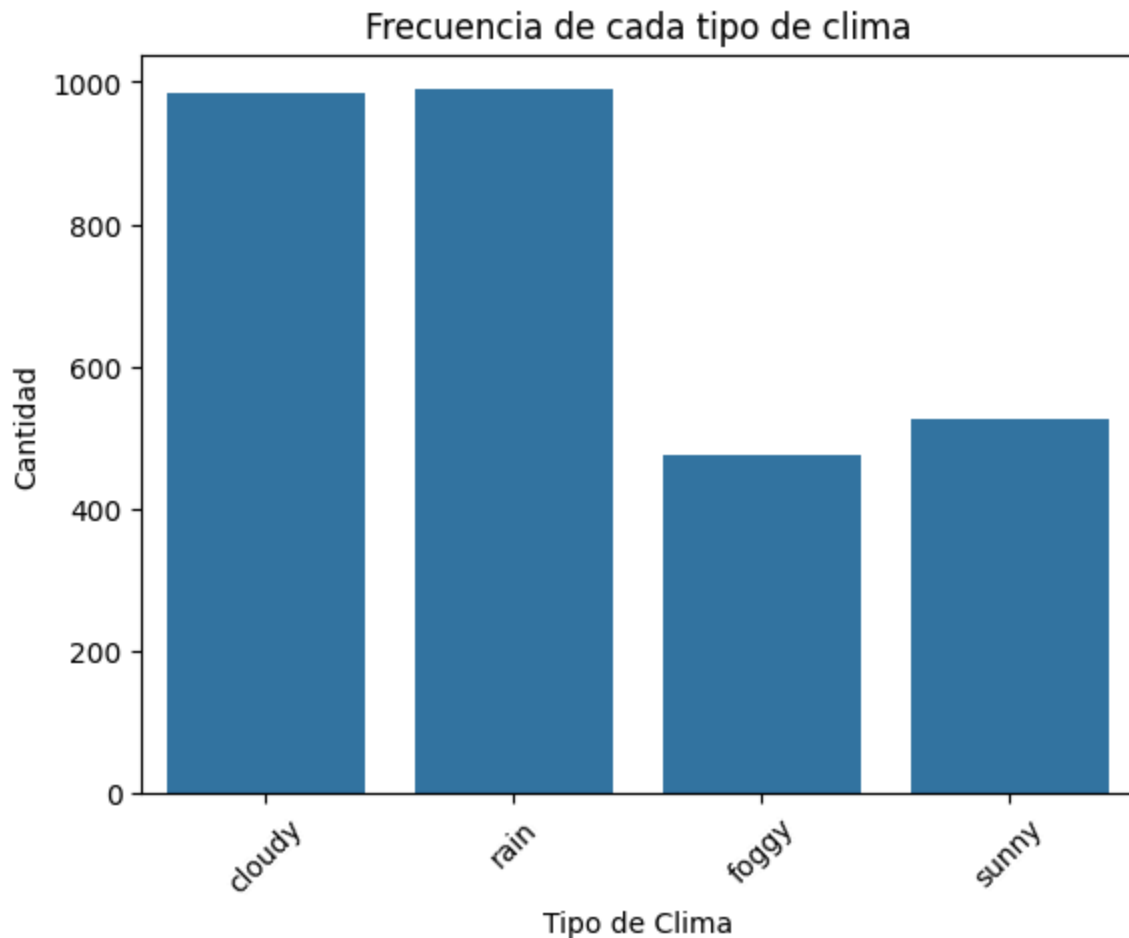
plt.title('Boxplot de conteo_buses')
plt.xlabel('conteo_buses')
plt.ylabel('frecuencia')
plt.show()
```



```
In [ ]: #Diagrama para la variable clima
sns.countplot(x='clima', data=data)

plt.title('Frecuencia de cada tipo de clima')
plt.xlabel('Tipo de Clima')
plt.ylabel('Cantidad')
plt.xticks(rotation=45)
plt.show()
```





In [ ]: `data_codificada.describe()`

Out [ ]:

|              | dia_mes     | conteo_carros | conteo_bicicletas | conteo_buses | conteo_camiones | tie   |
|--------------|-------------|---------------|-------------------|--------------|-----------------|-------|
| <b>count</b> | 2976.000000 | 2976.000000   | 2976.000000       | 2976.000000  | 2.976000e+03    | 2976. |
| <b>mean</b>  | 16.000000   | 68.715439     | 14.921681         | 15.284370    | 3.375539e+03    | 1.    |
| <b>std</b>   | 8.945775    | 45.812366     | 12.845334         | 14.339596    | 1.833086e+05    | 1.    |
| <b>min</b>   | 1.000000    | 6.000000      | 0.000000          | 0.000000     | 0.000000e+00    | 0.    |
| <b>25%</b>   | 8.000000    | 19.750000     | 5.000000          | 1.000000     | 6.000000e+00    | 0.    |
| <b>50%</b>   | 16.000000   | 64.000000     | 12.000000         | 12.000000    | 1.400000e+01    | 1.    |
| <b>75%</b>   | 24.000000   | 107.000000    | 22.000000         | 25.000000    | 2.325000e+01    | 2.    |
| <b>max</b>   | 31.000000   | 180.000000    | 70.000000         | 50.000000    | 1.000000e+07    | 3.    |

## Modelo de predicción

In [ ]: `from sklearn.model_selection import train_test_split`  
`from sklearn.linear_model import LogisticRegression`

```

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import confusion_matrix
from sklearn.metrics import precision_score
from sklearn.metrics import accuracy_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

```

```

In [ ]: #Separacion de datos. 80% para entrenar el modelo. 20% para la prediccion final.
train_size = int(len(data) * 0.8)
data_train = data_codificada.iloc[:train_size]
data_predict = data_codificada.iloc[train_size:]

data_train.count()

```

```

Out[ ]:

```

|                          |          |
|--------------------------|----------|
|                          | <b>0</b> |
| <b>dia_mes</b>           | 2380     |
| <b>conteo_carros</b>     | 2380     |
| <b>conteo_bicicletas</b> | 2380     |
| <b>conteo_buses</b>      | 2380     |
| <b>conteo_camiones</b>   | 2380     |
| <b>tiempo_2</b>          | 2380     |
| <b>dia_semana</b>        | 2380     |
| <b>clima</b>             | 2380     |
| <b>situacion_trafico</b> | 2380     |

**dtype:** int64

```

In [ ]: data_predict.count()

```

Out[ ]: **0**

|                          |     |
|--------------------------|-----|
| <b>dia_mes</b>           | 596 |
| <b>conteo_carros</b>     | 596 |
| <b>conteo_bicicletas</b> | 596 |
| <b>conteo_buses</b>      | 596 |
| <b>conteo_camiones</b>   | 596 |
| <b>tiempo_2</b>          | 596 |
| <b>dia_semana</b>        | 596 |
| <b>clima</b>             | 596 |
| <b>situacion_trafico</b> | 596 |

**dtype:** int64

```
In [ ]: #Se separan los datos con train_split
x = data_codificada.filter([ 'conteo_carros', 'conteo_bicicletas', 'conteo_buses',
y = data_codificada['situacion_trafico'].astype(int)

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_sta
```

```
In [ ]: x.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2976 entries, 0 to 2975
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   conteo_carros    2976 non-null   float64
1   conteo_bicicletas 2976 non-null   float64
2   conteo_buses     2976 non-null   float64
3   clima            2976 non-null   int8
dtypes: float64(3), int8(1)
memory usage: 72.8 KB
```

```
In [ ]: y.info()
```

```
<class 'pandas.core.series.Series'>
RangeIndex: 2976 entries, 0 to 2975
Series name: situacion_trafico
Non-Null Count  Dtype
-----
2976 non-null   int64
dtypes: int64(1)
memory usage: 23.4 KB
```

## Modelado con Regresión Logística

```
In [ ]: modelo = LogisticRegression()  
modelo.fit(X_train, y_train)  
y_pred_r1 = modelo.predict(X_test)
```

```
In [ ]: matriz_r1 = confusion_matrix(y_test, y_pred_r1)  
print('Matriz de confusión')  
print(matriz_r1)
```

Matriz de confusión  
[[177 24]  
 [ 18 377]]

```
In [ ]: precision_r1 = precision_score(y_test, y_pred_r1)  
print('Precisión del modelo')  
print(precision_r1)
```

Precisión del modelo  
0.940149625935162

```
In [ ]: recall_r1 = recall_score(y_test, y_pred_r1)  
print('Recall del modelo: ')  
print(recall_r1)
```

Recall del modelo:  
0.9544303797468354

```
In [ ]: f1_score_r1 = f1_score(y_test, y_pred_r1)  
print('F1 Score del modelo: ')  
print(f1_score_r1)
```

F1 Score del modelo:  
0.9472361809045227

```
In [ ]: accuracy = accuracy_score(y_test, y_pred_r1)  
print('Accuracy del modelo: ')  
print(accuracy)
```

Accuracy del modelo:  
0.9295302013422819

```
In [ ]: #Test de accuracy  
# Predicciones en entrenamiento y prueba  
y_train_pred = modelo.predict(X_train)  
y_test_pred = modelo.predict(X_test)  
  
# Accuracy en entrenamiento y prueba  
train_acc = accuracy_score(y_train, y_train_pred)  
test_acc = accuracy_score(y_test, y_test_pred)  
  
print(f"Accuracy en entrenamiento: {train_acc:.4f}")  
print(f"Accuracy en prueba: {test_acc:.4f}")
```

Accuracy en entrenamiento: 0.9210  
Accuracy en prueba: 0.9295

## Modelado con Random Forest

```
In [ ]: forest = RandomForestClassifier(bootstrap= True, max_depth= 10, max_features= 'sqrt')
forest.fit(X_train, y_train)
y_pred_rf = forest.predict(X_test)
```

```
In [ ]: matriz_rf = confusion_matrix(y_test, y_pred_rf)
print('Matriz de confusión')
print(matriz_rf)
```

Matriz de confusión  
[[200 1]  
[ 2 393]]

```
In [ ]: precision_rf = precision_score(y_test, y_pred_rf)
print('Precisión del modelo')
print(precision_rf)
```

Precisión del modelo  
0.9974619289340102

```
In [ ]: recall_rf = recall_score(y_test, y_pred_rf)
print('Recall del modelo: ')
print(recall_rf)
```

Recall del modelo:  
0.9949367088607595

```
In [ ]: f1_score_rf = f1_score(y_test, y_pred_rf)
print('F1 Score del modelo: ')
print(f1_score_rf)
```

F1 Score del modelo:  
0.9961977186311787

```
In [ ]: accuracy_rf = accuracy_score(y_test, y_pred_rf)
print('Accuracy del modelo: ')
print(accuracy_rf)
```

Accuracy del modelo:  
0.9949664429530202

## Prueba de sobreajuste para Random Forest

```
In [ ]: #Test de accuracy
# Predicciones en entrenamiento y prueba
y_train_pred = forest.predict(X_train)
y_test_pred = forest.predict(X_test)

# Accuracy en entrenamiento y prueba
train_acc = accuracy_score(y_train, y_train_pred)
test_acc = accuracy_score(y_test, y_test_pred)

print(f"Accuracy en entrenamiento: {train_acc:.4f}")
print(f"Accuracy en prueba: {test_acc:.4f}")
```

Accuracy en entrenamiento: 1.0000  
Accuracy en prueba: 0.9950

```
In [ ]: #Prueba de Validacion cruzada
from sklearn.model_selection import cross_val_score
cross_val_scores = cross_val_score(forest, X_train, y_train, cv=5, scoring='accuracy')
print(f"Accuracy promedio de validación cruzada: {cross_val_scores.mean():.4f}")
```

Accuracy promedio de validación cruzada: 0.9971

## Analisis final del mejor modelo

Se ha seleccionado el modelo hecho con Random Forest ya que su precision estuvo en un 99.6% de los datos, y aunque pueda ser muy alto, se comprobo mediante las pruebas de accuracy (incluyendo los datos de prueba y entrenamiento), que son valores muy similares, por lo tanto indica que no hay sobreajuste. Tambien se le hizo una prueba de Validacion cruzada la cual dio indices muy altos tambien, lo que indica que el modelo tiene un excelente rendimiento promedio.

En conclusion, a pesar de tener una precisión de 100% en el entrenamiento, eso no está generando un mal rendimiento en los datos nuevos, por lo tanto, el modelo generaliza bien.

```
In [ ]: #Ahora pruebo con nlos datos no involucrados en el entrenamiento, el 20% reservado
X_predict = data_predict.filter([ 'conteo_carros', 'conteo_bicicletas', 'conteo_bus
y_actual = data_predict['situacion_trafico']
```

```
In [ ]: # Predecir con el modelo Random Forest
y_pred = forest.predict(X_predict)
```

```
In [ ]: # Evaluar la precisión de los datos predichos
accuracy = accuracy_score(y_actual, y_pred)
print(f"Accuracy sobre el 20% reservado: {accuracy:.4f}")

# Matriz de confusión para comparar los datos reales y los predichos
print("\nMatriz de confusión:")
print(confusion_matrix(y_actual, y_pred))
```

Accuracy sobre el 20% reservado: 1.0000

Matriz de confusión:

```
[[210  0]
 [ 0 386]]
```

En conclusion, se ha creado un modelo casi perfecto, que no solo tienen un alto rendimiento en los datos de entrenamiento, sino que tambien tiene un excelente rendimiento en los datos nuevos que se le ingresan.

## Conversión de Jupyter a PDF

```
In [ ]: !jupyter nbconvert --to html "/content/Seguimiento_2.ipynb"
```